

READING SHEET

Side Effects — useEffect & useRef

Module 3 · Frontend: React & Next.js · Day 29 · Week 6

PROGRAM

CodeOps · Full Stack

WORK TYPE

Revision + mini-project

EST. TIME

2–3 hours

Tip How to use this sheet

This is your take-away for Day 29 — the day your components learn to reach outside themselves. It revises what a side effect is, how the dependency array and cleanup control an effect's lifecycle, how to fetch data properly, and what useRef is for. The Addis Eats menu stops being a hard-coded array and starts arriving from a server. Type every example: effects are much easier to understand when you have watched one run twice by accident.

Part 1 · Effects & useEffect

Rendering is supposed to be **pure**: props and state go in, JSX comes out, and nothing outside the component is touched. Anything that breaks that rule — a network request, a timer, setting `document.title`, writing to `localStorage`, subscribing to an event — is a **side effect**, and it belongs in `useEffect`.

Pure work versus effects

PURE RENDER WORK	SIDE EFFECT
Reading props and state	Fetching from an API
Calculating a total in ETB	Setting <code>document.title</code>
Filtering an array	Starting a timer or interval
Returning JSX	Reading or writing <code>localStorage</code>
Formatting a price	Subscribing to an event

The reason the split matters: React may render a component more than once for the same result. Pure work is harmless when repeated. A fetch is not.

Declaring an effect

`useEffect` takes two arguments — the function to run, and the array of values it depends on. React runs the function *after* the browser has painted the new render, and re-runs it only when something in the array has changed.

MenuTitle.jsx — useEffect

```
import { useEffect, useState } from "react";

function MenuTitle() {
  const [count, setCount] = useState(0);

  useEffect(() => {
    document.title = `Addis Eats — ${count} items`;
  }, [count]);
  // ^ the effect      ^ the dependency array

  return <button onClick={() => setCount(count + 1)}>Add</button>;
}
```

Warning The infinite loop

An effect with no dependency array runs after every render. If it also sets state, that state change causes another render, which runs the effect again — forever. React Strict Mode deliberately runs effects twice in development to surface exactly this class of bug early.

When not to use an effect

If a value can be calculated from props and state during render, calculate it. An effect that only turns data into other data adds an extra render and a value that can go stale — the same derived-value rule you met on Day 28.

derive during render, not in an effect

```
// wrong: an extra render and a value that can go stale
useEffect(() => setTotal(items.reduce(sum, 0)), [items]);

// right: just calculate it while rendering
const total = items.reduce(sum, 0);
```

Part 2 · Dependencies & Cleanup

The dependency array decides **how often** an effect runs. The function the effect returns decides **how it tidies up**. Between them they cover the whole lifecycle: mount, update and unmount.

SECOND ARGUMENT	WHEN THE EFFECT RUNS
[]	Once, after the first render only
[category]	After the first render, then whenever category changes
[a, b]	Whenever a or b changes
omitted	After every single render — rarely what you want

Every prop, state value or variable the effect reads belongs in the array. Leaving one out gives you a **stale closure** — the effect keeps working with the value it captured on an earlier render, and the bug is silent.

Cleanup

If an effect returns a function, React calls it before running the effect again and once more when the component is removed. Whatever the effect started, cleanup stops.

OrderTimer.jsx – start and stop

```
useEffect(() => {
  const id = setInterval(() => {
    setSeconds(s => s + 1);
  }, 1000);

  return () => clearInterval(id); // cleanup
}, []);

// Without the cleanup, every mount starts another
// interval and none of them ever stop.
```

MOMENT	WHAT REACT DOES
First render	Runs the effect
A dependency changed	Runs cleanup, then runs the effect again
Component removed	Runs cleanup one last time

Warning Leaks are silent

A forgotten interval or event listener keeps running after the component is gone. Nothing throws — the application just gets slower, and eventually React warns about a state update on a component that is no longer mounted.

Four mistakes you will meet this week

read the dependency array first

```
// 1. missing dependency – category is always "All" inside
useEffect(() => { load(category); }, []);

// 2. object recreated each render – never equal, so runs always
useEffect(() => { /* ... */ }, [{ id: 1 }]);

// 3. async on the effect – returns a promise, not a cleanup function
useEffect(async () => { /* ... */ }, []);

// 4. no cleanup – the listener outlives the component
useEffect(() => { window.addEventListener("resize", f); }, []);
```

Tip The async fix

Declare an async function inside the effect and call it: `async function load() { ... } load();` The effect itself must stay synchronous so it can still return a cleanup function.

Part 3 · Data Fetching

This is the everyday use of `useEffect`: loading data when a component appears. Real fetching means three things on screen — **loading**, **error** and **data** — and the early-return pattern from Day 27 is exactly what picks the right one.

Menu.jsx — load the Addis Eats menu

```
const [dishes, setDishes] = useState([]);
const [loading, setLoading] = useState(true);
const [error, setError] = useState(null);

useEffect(() => {
  async function load() {
    try {
      const res = await fetch("/api/dishes");
      if (!res.ok) throw new Error("Could not load the menu");
      setDishes(await res.json());
    } catch (e) { setError(e.message); }
    finally { setLoading(false); }
  }
  load();
}, []);
```

Warning fetch does not reject on 404

fetch only rejects when the network itself fails. A 404 or a 500 resolves normally, so you have to test `res.ok` yourself and throw — otherwise the error branch never runs and you try to render an error page as if it were a menu.

Showing the three states

early returns, in priority order

```
if (loading) return <p>Loading the menu...</p>;
if (error)   return <p className="err">{error}</p>;
if (dishes.length === 0) return <p>No dishes yet.</p>;

return dishes.map(d => <Dish key={d.id} {...d} />);
```

The order matters. Each guard means the code below it can assume the happy case. And an empty category is a *normal result*, not a failure — keep it out of the error branch or the person reading the screen will think something broke.

Refetching, and race conditions

Put category in the dependency array and React refetches whenever it changes. Cancel the previous request in cleanup, or a slow earlier response can land last and overwrite newer data.

Menu.jsx — category-driven fetching

```
useEffect(() => {
  const ctrl = new AbortController();
  setLoading(true);

  fetch(`/api/dishes?category=${category}`, { signal: ctrl.signal })
    .then(res => res.json())
    .then(setDishes)
    .catch(e => { if (e.name !== "AbortError") setError(e.message); })
    .finally(() => setLoading(false));

  return () => ctrl.abort();
}, [category]);
```

Tip Libraries do this for you — later

On Day 41 you meet SWR and React Query, which wrap caching, revalidation and cancellation into a single line. Write it by hand first, so you understand what they are doing on your behalf.

Part 4 · useRef

useRef returns an object with one property, `current`. React keeps that object identical across every render, but changing what is inside it never causes a render. It is a box that survives renders and that React otherwise ignores.

```

// useRef — one property: .current

const ref = useRef(initialValue);

ref.current          // read it
ref.current = newValue // write it — no re-render

// useState: changing the value redraws the component
// useRef:   changing the value redraws nothing

```

Reaching a DOM element

Pass a ref to a JSX element with the `ref` attribute and React puts the real DOM node into `.current` once it is on screen. That is why DOM work goes in an effect — before the paint, `current` is still null.

```

// OrderForm.jsx — focus on mount

function OrderForm() {
  const phoneRef = useRef(null);

  useEffect(() => {
    phoneRef.current.focus(); // the real <input>
  }, []);

  return <input ref={phoneRef} placeholder="09... or +2519..." />;
}

```

Choosing between useRef and useState

useState	useRef
Changing it re-renders	Changing it renders nothing
Read it during render	Read it in effects and handlers
Shown to the user	Kept behind the scenes
Replaced with a setter	Assigned with <code>.current =</code>
The order total in ETB	The interval id that updates it
The selected category	The input element to focus

Warning Never read a ref while rendering

If a value appears on screen it must be state. A ref changing will not trigger a render, so whatever the user is looking at would quietly go out of date. Refs are for bookkeeping the user never sees, and for real DOM nodes.

Assigned Reading

Required

1. [react.dev — Synchronizing with Effects](#) What / why: what effects are for, when they run, and how cleanup keeps them honest.
2. [react.dev — You Might Not Need an Effect](#) What / why: the cases where an effect is the wrong tool. Read this one twice — it prevents most effect bugs.

Optional — go deeper

- [Lifecycle of reactive effects](#) — setup and cleanup as a pair, and how dependencies drive both.
- [Referencing values with refs](#) — useRef for values that must survive renders without causing one.
- [Manipulating the DOM with refs](#) — focus, scroll and measure: the legitimate uses of a DOM ref.

Exercises

Work these in order and commit as you go. Each one connects your menu a little further to the world outside the component.

1. Set document.title to the number of dishes currently shown, updating whenever the list changes.
2. Move the menu array into public/dishes.json and fetch it in useEffect with an empty dependency array.
3. Add loading and error state, and render each one with an early return before the list.
4. Check res.ok and throw a clear message so a 404 reaches your error branch.
5. Add the category to the dependency array so switching category refetches the list.
6. Abort the previous request in cleanup, then confirm in the Network tab that it is cancelled.
7. Focus a search input on mount with useRef, and note why the focus call has to be inside an effect.

Mini-Project · The Menu That Loads Itself

Tip What you build

Turn the interactive menu from Day 28 into one that fetches its own data. The dishes arrive from an endpoint, the category filter drives a new request, every one of the loading, error and empty states is visible on screen, and switching category quickly never leaves the wrong dishes showing.

Requirements

- Dishes loaded in useEffect and stored in state — no hard-coded array left in the component.
- Separate loading and error state, each rendered with an early return.
- res.ok checked, with a message the user can actually understand.
- setLoading(false) in a finally block so a spinner can never get stuck.
- The category in the dependency array, so changing it refetches.
- AbortController cancellation in cleanup, verified in the Network tab.
- A search input focused on mount with useRef, and everything pushed to GitHub.

A worked start

Get one fetch working with all three states before adding the category dependency:

```
starter – fetch, loading, error

import { useEffect, useState } from "react";

export default function Menu() {
  const [dishes, setDishes] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    async function load() {
      try {
        const res = await fetch("/dishes.json");
        if (!res.ok) throw new Error("Could not load the menu");
        setDishes(await res.json());
      } catch (e) { setError(e.message); }
      finally { setLoading(false); }
    }
    load();
  }, []);

  if (loading) return <p>Loading the menu...</p>;
  if (error) return <p className="err">{error}</p>;

  return dishes.map(d => <Dish key={d.id} {...d} />);
}
```

What to submit

Your updated GitHub repository containing `src/Menu.jsx`, `src/api.js`, `src/DishList.jsx` and `public/dishes.json`, with a commit message describing what now comes from the server.

Check yourself

- Does the menu show a loading message before the dishes appear?
- If you rename `dishes.json`, do you get a clear error message instead of a blank screen?
- Does switching category trigger a new request in the Network tab?
- Is the previous request shown as cancelled when you switch quickly?
- Does the empty category show a friendly note rather than an error?
- Does the search field have focus as soon as the page loads?
- Is every value in the dependency array actually read by the effect — and nothing missing?